

Bài tập lý thuyết

Bài 1.

1. i chạy từ 10 đến $n + 5$, mỗi lần i tăng lên 2 đơn vị $\Rightarrow$ số lần hàm op được gọi:
($n - 5$)/2. big theta: $O(n)$
2. $O(\log n)$
3. $O(n)$
4. $O(n \log n)$
5. $O(n \log n)$
6. $O(n^3)$

Bài 2:

hàm op được gọi $\sim 2n^4 \log_2 n$ lần

Bài 3:

$O(n^3)$, $\Omega(\log n)$, $\Omega(n)$, $\Omega(n^2)$, $O(n^2)$

Bài 4:

4. COS226, Midterm s25, 1

Xét một cây nhị phân tìm kiếm được định nghĩa bởi cài đặt Java sau đây:

```
public class BST<Key extends Comparable<Key>, Value> {  
    private Node root;           // the root of the red-black BST  
    private int n;                // number of key-value pairs  
    ...  
    private class Node {  
        private Key key;         // the key  
        private Value value;     // the associated value  
        private Node parent;     // link to parent  
        private Node left;       // link to left subtree  
        private Node right;      // link to right subtree  
        private int count;       // number of nodes in subtree  
    }  
}
```

Handwritten notes:

- 16 (next to public class BST)
- 8x5 (next to private class Node)
- padding: 4 (next to private int n)
- padding: 4 (next to private class Node)
- inner class: 8 (next to private class Node)
- padding: 4 (next to private class Node)
- 16 8 4 } $\Rightarrow$ 32 (next to private int n)
- 4 } $\Rightarrow$ 72 (next to private class Node)

Một đối tượng BST sử dụng bao nhiêu bộ nhớ (tính bằng byte) theo kích thước n (số cặp key-value chứa trong bảng)?

Tính tất cả bộ nhớ (bao gồm cả tham chiếu đối tượng) mà đối tượng BST được cấp phát, nhưng không tính bộ nhớ cho các đối tượng String (mà client cấp phát). Sử dụng mô hình chi phí bộ nhớ 64-bit; đơn giản hóa câu trả lời bằng ký hiệu xấp xỉ ($\sim$).

Handwritten note: cây có n node $\Rightarrow 72n + 32$

Bài 5:

một 64-bit; đơn giản hóa câu trả lời bằng ký hiệu xấp xỉ ($\sim$).

5. **COS226, Midterm f24, 2a**

Đoạn code dưới đây in từ hello bao nhiêu lần? Giả sử n chia hết cho 100.

```
for (int i = 1; i <= n; i++)  
    for (int j = n; j >= i; j--)  
        for (int k = 1; k <= n; k = k + n/100)  
            System.out.println("hello");
```

$\Rightarrow \frac{100(n-1)}{n}$

$$n \times n \times \frac{100(n-1)}{n} \sim 100n^2$$

$\sim 200n$ $\sim 50n^2$ $\sim 100n^2$ $\sim \frac{1}{2} n^2 \log_{100} n$ $\sim n^3/100$ $\sim 100n^3$

Bài 6:

$\sim 200n$ $\sim 50n^2$ $\sim 100n^2$ $\sim \frac{1}{2} n^2 \log_{100} n$ $\sim n^3/100$ $\sim 100n^3$

6. **COS226, Midterm f24, 2b**

Chọn biểu thức mô tả đúng về hàm $n \log^2 n + 3n\sqrt{n} - 5n$

$\sim n\sqrt{n}$ $\Theta(n \log^2 n)$ $O(n^3)$ $O(n\sqrt{n})$ $O(n)$ $\Omega(\log n)$

Bài tập algorithm analystic

Bài 1:

với $n = 3$, ta thấy $S(n) = 3 * (3 - 1) * (3 - 2) / 6 = 1$.

và đúng là chỉ có 1 cách chọn bộ 3 số với $n = 3$.

giả sử đúng với $n = k$, ta có :

$$S(k) = k * (k - 1) * (k - 2) / 6.$$

xét 1 tập gồm $k + 1$ phần tử, số cách chọn ra 1 bộ 3 sẽ là:

$S(k) + kC2$ (tức là số cách chọn cho trường hợp $k + 1$ phần tử sẽ là số cách chọn cho trường hợp có k phần tử cộng với số cách chọn ra 2 phần tử để ghép với phần tử $k + 1$ để tạo thành 1 bộ 3)

$$S(k+1) = k*(k-1)*(k-2)/6 + k*(k-1)/2 = (k+1) * (k + 1 - 1) * (k + 1 - 2) / 6.$$

=> biểu thức đúng với $n = k + 1$

Bài 2:

Ta có thể tối ưu chương trình ThreeSum từ $O(n^3)$ về $O(n^2 \log n)$.

ta duyệt 2 vòng lặp lồng nhau để tìm tổng của mỗi bộ 2 số, đối với mỗi bộ hai số đó ta áp dụng tìm kiếm nhị phân để tìm ra giá trị $-(a[i] + a[j])$. Nếu tìm ra thì tức là với 2 số $a[i]$, $a[j]$ có tồn tại một giá trị $a[k]$ sao cho $a[i] + a[j] + a[k] = 0$.

Bài 5:

1.4.5. Cho biết kết quả áp dụng phép xấp xỉ cho các biểu thức dưới đây

- a. $N + 1 \sim N$
- b. $1 + 1/N \sim 1$
- c. $(1 + 1/N)(1 + 2/N) \sim 1$
- d. $2N^3 - 15N^2 + N \sim 2N^3$
- e. $\lg(2N)/\lg N \sim 1$
- f. $\lg(N^2 + 1)/\lg N \sim 2$
- g. $N^{100}/2^N \sim 0$

Bài 6:

a.

1.4.6 Cho biết bậc tăng của thời gian chạy (theo hàm của N) của mỗi đoạn code sau:

a.

```
int sum = 0;
for (int n = N; n > 0; n /= 2)
    for(int i = 0; i < n; i++) sum++;
```

$O(n)$

b.

```
int sum = 0;
for (int i = 1; i < N; i *= 2)
    for (int j = 0; j < i; j++) sum++;
```

$O(n)$

c.

```
int sum = 0;
for (int i = 1; i < N; i *= 2)
    for (int j = 0; j < N; j++) sum++;
```

$O(n \log n)$

Bài 7:

```
*/
public static int count(int[] a) {
    int n = a.length;
    int count = 0;
    for (int i = 0; i < n; i++) {
        for (int j = i+1; j < n; j++) {
            for (int k = j+1; k < n; k++) {
                if (a[i] + a[j] + a[k] == 0) {
                    count++;
                }
            }
        }
    }
    return count;
}
```

thuật toán three sum sử dụng 3 vòng lặp lồng nhau, chạy đến n theo cấp số cộng với công sai là 1 => tổng số lượng phép kiểm tra là $n*(n - 1)(n - 2) / 6$.
vậy độ phức tạp thuật toán là $O(n^3)$.

Bài 8:

```
public class bai8{
    public int count(int[] arr){
        int res = 0;
        int cur = 0;
        Array.sort(arr);
        for(int i = 1, i < arr.size(); i++){
            if(a[i] == a[i - 1]){
                cur++;
                res += cur;
            }
            else cur = 0;
        }
        return res;
    }
}
```

Bài 10:

```
public class bai10{
    public int binary_search(int[] arr, key){
        int ans = -1;
        int l = 0, r = arr.size() - 1;
        while (l <= r){
            int mid = (l + r) / 2;
            if(arr[mid] == key){
                ans = mid;
                r = mid - 1;
            }
            else if(arr[mid] > key) r = mid - 1;
            else l = mid + 1;
        }
    }
}
```

```

        return ans;
    }
}

```

Bài 12:

```

public class bai12{
    public void solve(int[] arr1, int[] arr2, int N){
        int i = 0, j = 0;
        while(i < N && j < N){
            if(a[i]< a[j]) {
                System.out.print(arr1[i], " ");
                i ++;
            }
            else {
                System.out.print(arr2[j], " ");
                j ++;
            }
        }
        while(i < N){
            System.out.print(arr1[i]);
            i ++;
        }
        while(j < N){
            System.out.print(arr2[j]);
            j ++;
        }
    }
}

```

Bài 13:

a. accumulator

+object overhead tốn 16 bytes

+biến n kiểu int => tốn 4 bytes

+biến sum, mu kiểu double => tốn 8 bytes

-> cộng tổng lại: 36 bytes, cộng thêm 4 bytes padding nữa tổng cộng sẽ là 40 bytes

b.transaction

theo shallow memory:

+object overhead tốn 16 bytes

+date, string, double đều tốn 8 bytes => 24 bytes

->tổng cộng 40 bytes

d. Point 2D

+object overhead tốn 16 bytes

+ 2 biến x,y kiểu double-> tốn 16 bytes

-> tổng cộng 32 bytes

e. Interval1D

object over head và 2 biến kiểu double -> tổng 32 bytes

f.Interval2D

object over head và 2 biến kiểu double -> tổng 32 bytes

Bài 14:

- thuật toán không tối ưu : 4 vòng for lồng nhau $O(n^4)$
- thuật toán tối ưu : $O(n^2 \log n)$

```
class Solution {
public:
    vector<vector<int>> fourSum(vector<int>& nums, int target) {
        vector<vector<int>> ans;
        if (nums.size() < 4) return ans;

        sort(nums.begin(), nums.end());

        int n = nums.size();
        for (int i = 0; i < n - 3; i++) {
            if (i > 0 && nums[i] == nums[i - 1]) continue;

            for (int j = i + 1; j < n - 2; j++) {
                if (j > i + 1 && nums[j] == nums[j - 1]) continue;

                int l = j + 1;
                int r = n - 1;

                while (l < r) {
                    long long sum = (long long)nums[i] + nums[j] +
nums[l] + nums[r];

                    if (sum > target) {
                        r--;
                    } else if (sum < target) {
                        l++;
                    } else {
                        ans.push_back({nums[i], nums[j], nums[l],
nums[r]});

                        while (l < r && nums[l] == nums[l + 1]) l++;
                        while (l < r && nums[r] == nums[r - 1]) r--;
                    }
                }
            }
        }
    }
};
```

```
                l++;  
                r--;  
            }  
        }  
    }  
    return ans;  
}  
};
```